

ColorDetector API 调用文档

一、API 概述

本模块提供颜色目标检测的标准化接口，支持通过编程方式集成到各类系统（如视觉流水线、机器人控制、自动化检测平台）。所有接口均遵循强类型定义，返回结构化数据，便于下游逻辑处理。

二、核心 API 清单

API 类型	接口名称	适用场景
类接口	ColorDetector	需多次复用检测器、维护状态（统计/配置）的场景
函数接口	detect_color_with_mask	单次快速检测、无状态调用的场景

三、类接口：ColorDetector

1. 初始化 API

```
from color import ColorDetector

detector = ColorDetector(
    target_color: str = "red",          # 必填：目标颜色
    min_area: int = 100,                # 可选：最小检测面积（像素）
    confidence_threshold: float = 0.3, # 可选：置信度阈值（0~1）
    enable_morphology: bool = True,    # 可选：是否启用形态学优化
    morph_kernel_size: int = 5         # 可选：形态学核大小（奇数）
)
```

参数约束

参数	类型	取值范围	默认值	说明
target_color	str	red / green / blue / yellow / orange / purple	"red"	颜色名称需全小写

<code>min_area</code>	<code>int</code>	≥ 0	<code>100</code>	小于该面积的区域将被过滤
<code>confidence_threshold</code>	<code>float</code>	[0.0, 1.0]	<code>0.3</code>	低于该值的轮廓将被丢弃
<code>enable_morphology</code>	<code>bool</code>	True / False	<code>True</code>	开启后自动去噪、填充空洞
<code>morph_kernel_size</code>	<code>int</code>	正奇数（如3/5/7）	<code>5</code>	核越大，去噪越强但可能丢失小目标

2. 检测 API（根据输入类型选择）

（1）BGR 图像检测

```
detections, mask = detector.detect_from_bgr(  
    bgr_image: np.ndarray, # 输入 BGR 图像（OpenCV 默认格式）  
    verbose: bool = False  # 是否输出调试日志（生产环境建议关闭）  
) -> Tuple[List[Dict], np.ndarray]
```

（2）HSV 图像检测

```
detections, mask = detector.detect(  
    hsv_image: np.ndarray, # 输入 HSV 图像（需提前转换）  
    verbose: bool = False  
) -> Tuple[List[Dict], np.ndarray]
```

3. 返回数据结构

(1) `detections`：检测结果列表

每个元素为**标准化目标字典**，可直接序列化（JSON）：

```
{  
    "id": 1,                // 目标编号（从1开始）  
    "color": "red",         // 检测颜色  
    "center_x": 320.5,      // 中心X坐标（像素）  
    "center_y": 240.0,      // 中心Y坐标（像素）  
    "width": 50.2,          // 最小外接矩形宽度（像素）  
    "height": 30.8,         // 最小外接矩形高度（像素）  
    "area": 1200.0,         // 目标面积（像素）  
    "aspect_ratio": 1.63,   // 宽高比（width/height）  
    "circularity": 0.85,    // 圆形度（0~1，越接近1越圆）  
    "confidence": 0.85,     // 置信度（等于圆形度）  
    "angle": 15.2           // 旋转角度（度，-90°~90°）  
}
```

(2) `mask`：二值掩码

-

类型：`np.ndarray`（`dtype=uint8`）

-

尺寸：与输入图像一致

-

取值：检测区域为 `255`，其余为 `0`

-

用途：可直接用于图像融合、ROI提取、可视化叠加

4. 统计 API（可选）

```
# 获取累计统计信息
stats = detector.get_statistics()
# 返回示例：
{
    "total_detections": 15,           # 累计检测目标总数
    "avg_processing_time_ms": 12.3,  # 平均单帧处理时间（毫秒）
    "config": {                      # 当前配置快照
        "target_color": "red",
        "min_area": 100,
        "confidence_threshold": 0.3
    }
}

# 重置统计数据（如切换场景时）
detector.reset_statistics()
```

四、函数接口：快速调用（无状态）

适用于单次检测、无需维护状态的场景：

```
from color import detect_color_with_mask

detections, mask = detect_color_with_mask(
    image: np.ndarray,           # 输入图像（BGR 或 HSV，自动识别）
    target_color: str = "red",  # 目标颜色
    min_area: int = 100,        # 最小面积
    verbose: bool = False       # 调试日志开关
)
```

自动识别逻辑

输入图像形状	处理方式
(H, W, 3)	视为 BGR 图像，自动转 HSV
(H, W)	视为 HSV 图像，直接检测

五、API 调用示例

示例1：集成到视觉流水线

```
from color import ColorDetector
import cv2

# 1. 初始化检测器（全局单例）
detector = ColorDetector(
    target_color="blue",
    min_area=50,
    confidence_threshold=0.5
)

def process_frame(frame: np.ndarray) -> dict:
    """处理单帧图像，返回结构化结果"""
    detections, mask = detector.detect_from_bgr(frame, verbose=False)

    # 提取关键信息（供下游使用）
    result = {
        "timestamp": time.time(),
        "target_count": len(detections),
        "targets": [
            {
                "x": det["center_x"],
                "y": det["center_y"],
                "confidence": det["confidence"]
            }
            for det in detections
        ]
    }
    return result
```

示例2：批量图像处理

```
import glob
from color import detect_color_with_mask

image_paths = glob.glob("dataset/*.jpg")
results = []

for path in image_paths:
    img = cv2.imread(path)
    detections, _ = detect_color_with_mask(img, target_color="green")

    results.append({
        "image": path,
        "detections": detections
    })
```

六、错误处理规范

异常场景	API 行为	处理建议
不支持的颜色	返回空列表 + 零掩码	调用前校验 <code>target_color in ["red", "green", ...]</code>
输入图像为空	抛出 <code>ValueError</code>	调用前检查 <code>image is not None</code>
图像通道数错误	抛出 <code>AssertionError</code>	确保输入为 3 通道（BGR）或 1 通道（HSV）

七、扩展 API（自定义颜色）

如需支持新颜色，可通过修改类属性扩展：

```
detector = ColorDetector(target_color="custom")
detector.color_ranges["custom"] = [
    [[lower_h, lower_s, lower_v], [upper_h, upper_s, upper_v]]
]
```

八、性能参考

图像分辨率	平均处理时间	内存占用
-------	--------	------

640×480	~8 ms	~10 MB
1280×720	~15 ms	~20 MB
1920×1080	~25 ms	~35 MB